

AFL Assistant

Complete Project Report — Task 1 to Task 5 AI-Powered AFL Question Answering, Retrieval, Prediction & Monitoring System

Introduction

The AFL Assistant project was developed as an end-to-end artificial intelligence system focused on Australian Football League information, statistical retrieval, conversational interaction, and machine-learning-based prediction. The purpose of the project was to move beyond a simple chatbot and build a structured assistant that can understand different types of AFL requests, route them to the appropriate component, retrieve information from AFL datasets, execute prediction models, maintain conversational context, protect itself against off-topic and prompt-injection requests, and expose its functionality through an API.

The system combines conversational AI with structured AFL datasets and previously trained prediction models. Intent-based routing separates factual questions, retrieval requests, prediction requests, and off-topic requests. LangGraph is used to orchestrate the workflow, while the prediction component uses trained machine-learning models. Team-name resolution was implemented so common AFL aliases such as Pies, Cats, Tigers, Blues, Bombers, Hawks, Swans, Lions, Dockers, Eagles, Bulldogs, Saints, Demons, Kangaroos, Crows, Power, Giants, and Suns can be mapped to the corresponding dataset team names.

Task 1 — System Hardening

The first task focused on making the complete AFL Assistant pipeline safer and more reliable. Consistent error handling was introduced so invalid inputs and failures do not result in uncontrolled exceptions being exposed to users. The prediction pipeline validates both participating teams before model execution. Unknown AFL teams return a controlled error, while selecting the same team twice is rejected as an invalid matchup. Team aliases are resolved before the prediction model is called.

Timeout handling was introduced for prediction operations so a slow prediction call does not block the complete application indefinitely. A standard prediction disclaimer was applied throughout prediction responses: "The predicted probability is a model estimate, not a certainty." This ensures model output is communicated as a probability rather than as a guaranteed match result.

Security and scope protection were another major part of Task 1. Prompt-injection detection was added to identify attempts to override the assistant's instructions. The assistant was designed to remain within the AFL domain, and off-topic requests are rejected or redirected. Multiple prompt-injection-style test cases were executed to confirm that the system does not simply follow instructions attempting to change its intended scope. Basic abuse tracking was also introduced.

Task 1 Conclusion: The system-hardening stage established the safety and reliability foundation of the AFL Assistant. Validation, timeout handling, consistent error responses, prediction disclaimers, scope protection, prompt-injection detection, and abuse tracking make the system more controlled and suitable for subsequent API and deployment work.

Task 2 — Comprehensive Evaluation

The second task focused on evaluating the complete assistant rather than testing only individual components. A combined evaluation framework was created around factual AFL question answering, prediction testing, scope and safety guardrails, and multi-turn conversational coherence. Each test case is stored with its category, query, expected behavior, actual result, and pass or fail status so individual cases can be reviewed.

Factual evaluation included checks related to AFL teams, team records, player information, and available dataset fields. Prediction evaluation checked whether valid team combinations could reach the prediction model and whether invalid inputs were handled safely. Scope testing included ordinary off-topic questions and prompt-injection-style requests designed to make the assistant ignore its AFL-only instructions. Multi-turn tests checked whether the conversation identifier and stored history could support follow-up interactions.

The evaluation process also introduced identification of the weakest category rather than relying only on one overall score. This makes it possible to identify where the next development effort should be concentrated. The evaluation table can be exported as a CSV file for further analysis and inclusion in the final project deliverables.

Task 2 Conclusion: The evaluation stage provided a structured way to measure the behavior of the complete AFL Assistant across accuracy, prediction, safety, and conversation scenarios. The test-table approach makes failures traceable and provides a foundation for future evaluation expansion. Additional real-world prediction measurements and benchmark comparison can further strengthen this evaluation.

Task 3 — FastAPI API and Structured Logging

The third task converted the AFL Assistant into an API-accessible application using FastAPI. The API accepts a message and a conversation identifier through a chat endpoint. The conversation identifier allows the application to maintain a separate conversation history for each user or session, providing the basic structure required for multi-turn interaction through an external client or user interface.

The API response contains the conversation identifier, generated response, detected intent, prediction metadata when applicable, tools called, response latency, and token usage when available. A health endpoint was also added so the service can be checked independently of normal chat processing. The API was tested locally and can be exposed through the FastAPI Swagger documentation interface.

Structured logging records important operational information including timestamp, conversation identifier, original query, detected intent, tools called, latency, token usage, and request status. This allows developers to understand how the system processed individual requests rather than seeing only the final response.

Task 3 Conclusion: The API stage transformed the assistant from a notebook-oriented prototype into a service-oriented application. FastAPI provides a clear interface for clients, while conversation storage and structured logs provide the foundation for debugging, monitoring, and future production integration.

Task 4 — Monitoring and Weekly Model Refresh

The fourth task addressed what happens after the system is deployed. A monitoring framework was created around response latency, tool error rate, off-topic leak rate, prediction accuracy drift, model freshness, and evaluation stability. Explicit alert thresholds were defined so operational problems can be detected consistently.

The monitoring configuration uses a response-latency alert threshold of more than five seconds, a tool-error alert threshold above five percent, and an off-topic leak alert threshold above two percent. Prediction accuracy is reviewed weekly, with more than five percent degradation from the established baseline treated as an alert condition.

A weekly refresh workflow was designed. Newly completed AFL matches are added to the historical dataset, team and player feature tables are updated, data types and missing values are checked, and potential data leakage is reviewed. The existing model is evaluated before retraining. When sufficient new data is available, the model can be retrained and compared with the previous model. The previous version is retained as a rollback option.

Task 4 Conclusion: The monitoring stage established a repeatable operational process for maintaining the AFL Assistant after deployment. It connects system monitoring with model-quality monitoring and provides a weekly refresh strategy so the prediction system can evolve as new AFL rounds are completed.

Task 5 — Final Deliverables and Presentation

The fifth task packaged the completed project into stakeholder-ready deliverables. The final system includes the LangGraph application, FastAPI API, prediction model integration, AFL retrieval components, conversation memory, safety guardrails, evaluation framework, monitoring checklist, weekly refresh workflow, executive report, and demonstration script.

A five-to-seven-minute demonstration was designed around four scenarios. The first demonstrates a factual AFL question such as “Tell me about the Pies.” The second demonstrates a match prediction such as “Predict Pies vs Cats.” The third demonstrates an off-topic request to confirm the domain boundary. The fourth demonstrates a prompt-injection attempt to verify that the system does not allow user instructions to override its AFL-only scope.

Task 5 Conclusion: The final deliverable stage converted the technical work into a complete project package suitable for demonstration and evaluation. The executive report explains the system at stakeholder level, while the presentation and demo flow provide a concise way to demonstrate the most important capabilities.

Overall Conclusion

The AFL Assistant project demonstrates an end-to-end AI application that combines conversational artificial intelligence, structured AFL datasets, machine-learning prediction, LangGraph orchestration, FastAPI deployment, safety guardrails, structured logging, monitoring, and model maintenance. The project was developed progressively from system hardening and evaluation through API integration and operational monitoring.

A major strength of the architecture is the separation between conversational processing, retrieval, prediction, and safety control. This separation makes the system easier to test and maintain while reducing the risk that an unrelated request will be processed as an AFL query or that an unsupported prediction will be presented as a fact. Team-name resolution improves usability, while structured error handling and prediction disclaimers make model interaction more transparent.

The project also establishes a practical foundation for future development. The prediction system can be evaluated against stronger public or historical benchmarks, the retrieval layer can be expanded with additional AFL statistics, and automated data ingestion can keep the system current as new matches are completed. Production deployment can further improve persistent conversation storage, process-level timeout handling, monitoring dashboards, automated alerts, and model version management.

Overall, the completed project provides a practical example of how an AI assistant can be built as a complete software system rather than as an isolated language-model interface. It combines domain grounding, machine learning, orchestration, API engineering, safety, evaluation, and continuous improvement into one integrated AFL-focused application.